

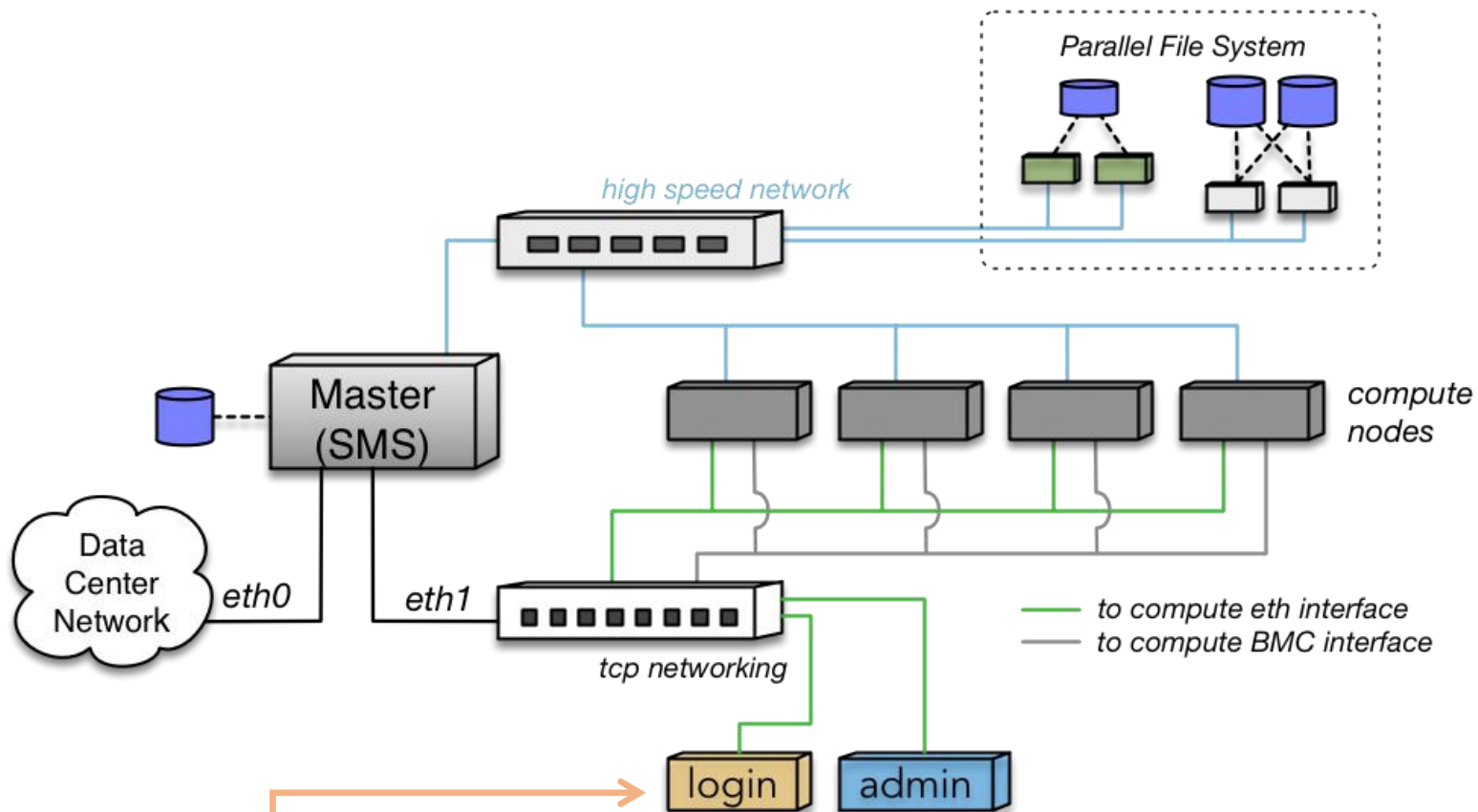
集群用户培训

Version 0.5

主要内容

- 计算集群资源概况
- 使用集群需要用到的基本linux命令
- 软件管理系统 module
- 作业管理系统 slurm

计算集群架构



高性能计算硬件资源

PARTITION	NODES	Single Node Configuration	Total Cores	NOTE
4T64c	4	4 T memory	256	Fat Nodes (2021)
		2*CPU (Platinum8358, 32c)		
4T56c	9	4 T memory	504	Fat Nodes (2021)
		2*CPU (Gold 6330, 28c)		
256G56c	75	2*CPU (Gold 6330, 28c)	4200	new Nodes (2021)
512G56c	1	2*CPU (Gold 6330, 28c)	56	(2021)
128G24c	12	128 G memory	288	(2015)
		2*CPU (E5-2680v3, 12c)		
64G24c	12	64 G memory	288	(2015)
		2*CPU (E5-2680v3, 12c)		
256G16c1080Ti	2	256 G memory 4*1080Ti GPU	32	GPU Nodes (2018)
		2*CPU(E5-2620 v4, 8c)		
768G128cAMD	8	768 G memory	1024	Partition of Professor Yue's group (2024)
		2*CPU(AMD 9554, 64c)		

使用集群需要用到的基本linux命令

1. 文件夹操作命令：

文件夹切换 cd;目录查看 ls;目录操作(增mkdir，删rm，改mv 复制cp，查find);

2. 文件查看修改命令 cat tail short head vim

3. 文件传输 scp wget

4. conda常用命令

使用集群需要用到的基本linux命令

1. 文件夹操作命令：

文件夹切换 cd;目录查看 ls;目录操作(增mkdir，删rm，改mv 复制cp，查find);

```
[]$ cd /home/example/
```

```
[example]$ ls
```

```
Academic_and_Research_Computing_Services.pdf fedml gym lumerical matlab namd  
stata vasp
```

```
[example]$ cp -r gym/ ~/
```

```
[example]$ cd ~/gym/
```

```
[gym]$ ls
```

```
gym.sbatch work.py
```

```
[gym]$ mkdir test
```

```
[gym]$ ls
```

```
gym.sbatch test work.py
```

```
[gym]$ rm -r test/
```

```
[gym]$ mv gym.sbatch test.sbatch
```

```
[gym]$ cat test.sbatch
```

使用集群需要用到的基本linux命令

2. 文件查看修改命令 cat tail short head vim

```
#!/bin/bash
#SBATCH --job-name vaspWork
#SBATCH --partition 64G24c # 128G24c
#SBATCH --qos=ug_qos
#SBATCH --ntasks 112      #number of cores: 28 56 112
##SBATCH --exclusive      #Exclusive execution mode, when use 56, 112... cores.
###
ulimit -s unlimited
ulimit -l unlimited
#
module use /share/base/modulefiles/tools
module use /share/base/modulefiles/libraries
module use /share/base/modulefiles/parallel
module use /share/base/modulefiles/compilers
#
module load vasp/5.4.4.E5-2680v3
mpirun vasp_ncl > $SLURM_JOBID.log 2>&1
```

使用集群需要用到的基本linux命令

version 1.1
April 1st, 06
翻译:2006-5-21

vi / vim 键盘图

Esc
命令
模式

~ 转换 大小写	! 外部 过滤器	@ 运行 宏	# prev ident	\$ 行尾	% 括号 匹配	^ "软" 行首	& 重复 :s	* next ident	(句首	) 下一 句首	"soft" bol down	+ 后一行 行首
\. 跳转到 标注	1	2	3	4	5	6	7	8	9	0 "硬" 行首	- 前一行 行首	= 自动 格式化
Q 切换至 ex模式	W 下一 单词	E 词尾	R 替换 模式	T back till	Y 拷贝 行	U 撤消 行内命令	I 到行首 插入	O 分段 (前)	P 粘贴 (前)	{ 段首	}	段尾
q 录制 宏	w 下一 单词	e 词尾	r 替换 字符	t till	y 拷贝	u 撤消 命令	i 插入 模式	o 分段 (后)	p 粘贴 (后)	[杂项	]	杂项
A 在行尾 附加	S 删除行 并插入	D 删除 至行尾	F 行内字符 反向查找	G 文尾/ 行号	H 屏幕 顶行	J 合并 两行	K 帮助	L 屏幕 底行	:	ex 命令	" 寄存器 标识	行首/ 列
a 附加	s 删除字符 并插入	d 删除	f 行内字符 查找	g 附加 命令	h ←	j ↓	k ↑	l →	;	重复 t/T/f/F	' 跳转到标 注的行首	\ 未用!
Z 退出	X 退格	C 修改 至行末	V 可视 行模式	B 前一 单词	N 查找 上一处	M 屏幕 中间行	< 反缩进	> 缩进	?	向前 搜索		
Z 附加 命令	x 删除 (字符)	c 修改	v 可视 模式	b 前一 单词	n 查找 下一处	m 设置 标注	, 反向 t/T/f/F	. 重复 命令	/	向后 搜索		

动作	移动光标, 或者定义操作的范围
命令	直接执行的命令, 红色命令 进入编辑模式
操作	后面跟随表示操作范围的指令
extra	特殊功能, 需要额外的输入
q.	后跟字符参数

w,e,b 命令

小写(b): quux(foo, bar, baz);
大写(B): QUUX(FOO, BAR, BAZ);

主要ex命令:

:w (保存), :q (退出), :q! (不保存退出)
:e f (打开文件 f),
:%s/x/y/g ('y' 全局替换 'x').
:h (帮助 in vim), :new (新建文件 in vim),

其它重要命令:

CTRL-R: 重复 (vim),
CTRL-F/-B: 上翻/下翻,
CTRL-E/-Y: 上滚/下滚,
CTRL-V: 块可视模式 (vim only)

可视模式:

漫游后对选中的区域执行操作 (vim only)

备注:

- (1) 在 拷贝/粘贴/删除 命令前使用 "x (x=a..z,*) 使用命令的寄存器('剪贴板') (如: "ay\$ 拷贝剩余的行内容至寄存器 'a')
- (2) 命令前添加数字 多遍重复操作 (e.g.: 2p, d2w, 5i, d4j)
- (3) 重复本字符在光标所在行执行操作 (dd = 删除本行, >> = 行首缩进)
- (4) ZZ 保存退出, ZQ 不保存退出
- (5) zt: 移动光标所在行至屏幕顶端, zb: 底端, zz: 中间
- (6) gg: 文首 (vim only), gf: 打开光标处的文件名 (vim only)

使用集群需要用到的基本linux命令

3.压缩、解压和打包文件：tar 使用tar压缩文件

```
tar -zcvf test.tar.gz ./test/
```

该命令表示压缩当前文件夹下的文件夹test，压缩后缀名为test.tar.gz

如果不需要压缩成gz，只需要后缀为tar格式的，那么输入如下命令：

```
tar -cvf test.tar ./test/
```

使用tar解压文件

```
tar -xzvf test.tar.gz
```

该命令表示把后缀为.tar.gz的文件解压到当前文件夹下。

如果压缩文件的后缀是.tar，没有gz，则使用命令：

```
tar -xvf test.tar
```

登录集群

login nodes:

10.20.39.6 login1

10.20.54.82 login2

10.20.26.130 login3

10.20.8.67 login4

172.18.6.51 login5 (for 24c partitions, 在24c分区使用的软件可以在计算节点或此登录节点编译)

举例:

登录: ssh username@10.20.39.6

传输文件: ssh file1 file2 username@10.20.39.6:~/

传输文件夹:ssh -r dir1 dir2 username@10.20.39.6:~/

软件环境管理工具

Lmod的使用介绍



https://lmod.readthedocs.io/en/latest/010_user.html

如何在集群中使用软件和配置环境变量？

个人电脑方案：使用shell配置,写入脚本或者写入~/.bashrc 或者 ~/.bashr_profile文件

```
export PATH=/.../path-to-softwares-dir/bin:$PATH
```

```
export LIBRARY_PATH=/.../path-to-softwares-dir/lib:$LIBRARY_PATH
```

```
export LD_LIBRARY_PATH=/.../path-to-softwares-dir/lib:$LD_LIBRARY_PATH
```

```
export C_INCLUDE_PATH (for C header files) or CPLUS_INCLUDE_PATH
```

```
export MAN_PATH
```

```
export
```

.....

设置后在环境中无法灵活调整

变量多且长，软件更新变更后无法管理

```
[local0tanghong@ln01 julia-1.7.0-beta4]$ tree -L 1
```

```
.
├── bin
├── etc
├── include
├── lib
├── libexec
├── LICENSE.md
└── share
```

modules 程序通过modulefile文件提供可以动态修改的用户环境。
便于使用不同的软件和避免切换相同软件的不同版本时候发生冲突。

常用命令：

查看可用的软件

module avail (module av)

module av

```
----- /share/base/modulefiles/compilers -----  
anaconda/2 anaconda/3 conda intel/2017.8 intel/2018.4 intel/2019.5 intel/2022.0.1  
  
----- /share/base/modulefiles/parallel -----  
advisor/latest      dev-utilities/latest  intel_ipp_ia32/latest  mpi/2021.6.0  
ccl/2021.6.0        dnnl-cpu-gomp/latest  intel_ipp_intel64/2021.6.0  mpi/mpi/2021.5.0  
ccl/latest          dnnl-cpu-iomp/latest  intel_ipp_intel64/latest  mpi/latest
```

...

加载某一个modulefile 用于设置环境以便于使用相应的软件

module load <package>

例如： *module load anaconda*

module load cuda 加载的是哪个环境变量？

查看当前加载的软件

module list

卸载已经加载的某些软件

module unload package1 package2 ...

或

module rm package1 package2

卸载所有已经加载的软件

module purge

\$ module purge; module load gcc python; module list

Currently Loaded Modulefiles:

1) python/3.9.6 2) cuda/11.4

modulefile 文件介绍

```
$ pwd
/share/base/modulefiles/tools/vasp
$ ls
5.4.4  5.4.4.E5-2680v3 5.4.4.vtst.178.intel2022 6.1 6.3.0.nvhpc.23.7 6.4.0.oneapi
5.4.4.c 5.4.4.omc      5.4.4.wannier90_2.1      6.3 6.3.0.wannier90
6.4.0.oneapi.O3
$ cat 5.4.4.E5-2680v3
#%Module
module-whatis "Enable usage for VASP version 5.4.4"
conflict vasp
setenv VASP_DIR /share/software/vasp/5.4.4/E5-2680v3
module load intel/2022.0.1
module load mkl/2022.0.1
module load mpi/impi/2021.5.0
prepend-path PATH          $env(VASP_DIR)
```

集群中modulefile文件夹的设置

```
$ echo $MODULEPATH  
/share/base/modulefiles/compilers:/share/base/modulefiles/parallel:/share/base/modulefiles/libraries:/share/base/modulefiles/tools:/share/software/modules/5.0.1/modulefiles  
$ tail -f -n 3 /etc/profile  
module use /share/base/modulefiles/libraries  
module use /share/base/modulefiles/parallel  
module use /share/base/modulefiles/compilers
```

个人专用的module 环境设置

```
export MODULEPATH=/.../path-to-your-modulefiles-dir:$MODULEPATH  
OR  
module use /.../path-to-your-modulefiles-dir
```

写入作业执行文件

练习

使用module调用python
输入python -V 查看版本
再调用anaconda3
再次输入python -V查看版本

```
[root@login2 vasp]# module purge
[root@login2 vasp]# module load python
[root@login2 vasp]# python --version
Python 2.7.18
[root@login2 vasp]# which python
/share/software/python/2.7.18/bin/python
[root@login2 vasp]# module rm python
[root@login2 vasp]# module load anaconda
[root@login2 vasp]# python --version
Python 3.9.7
[root@login2 vasp]# which python
/share/software/anaconda3/bin/python
```

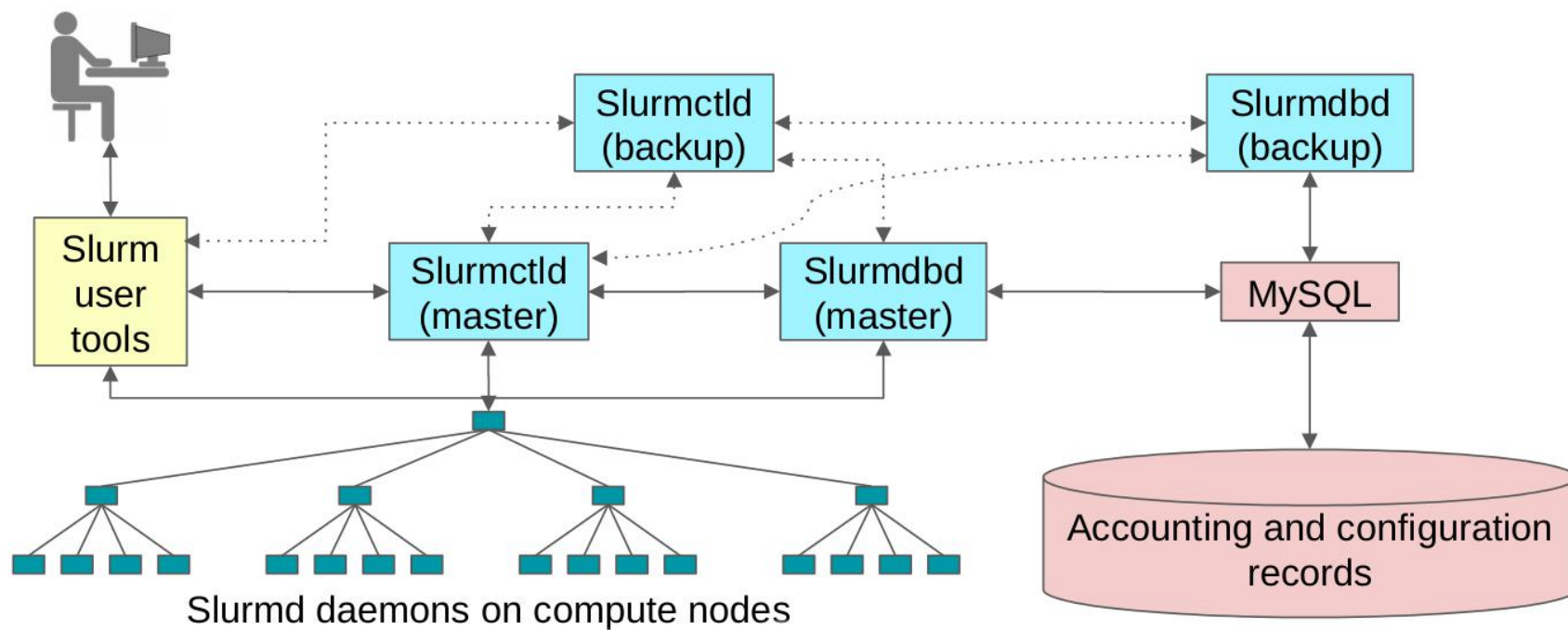
高性能计算集群任务调度工具

SLURM的使用介绍



Simple Linux Utility for Resource Management

slurm工作流程



Copyright 2018 SchedMD LLC

任务排队	优先级调度
Torque	Maui
Slurm	
Lsf	
pbspro	

SLURM的常用命令

- `sbatch jobscript.sbatch` 提交任务脚本排队执行 (批处理模式)

\$ sbatch stata.sbatch

Submitted batch job 1630

- `scancel jobID` 取消正在排队或者运行的任务

\$ scancel 1630

- `bjobs` 查看任务情况/jobID
- `scontrol show job jobID` 查看任务记录

sbatch 脚本中的参数

```
#!/bin/bash
#SBATCH --job-name vaspWork
#SBATCH --partition 64G24c 128G24c
#SBATCH --qos=ug_qos .....
#SBATCH --ntasks 24          #number of cores: 28 56 112
###SBATCH --exclusive        #Exclusive execution mode, when use 56, 112... cores.
###
ulimit -s unlimited
ulimit -l unlimited
#
module use /share/base/modulefiles/tools
module use /share/base/modulefiles/libraries
module use /share/base/modulefiles/parallel
module use /share/base/modulefiles/compilers
#
module load vasp/5.4.4.E5-2680v3
mpirun vasp_ncl > $SLURM_JOBID.log 2>&1
```

常用参数

--help # 显示帮助信息;
-A <account> # 指定计费账户;
-D, --chdir=<directory> # 指定工作目录;
--get-user-env # 获取当前的环境变量;
--gres=<list> # 使用gpu这类资源, 如申请两块gpu则--gres=gpu:2
-J, --job-name=<jobname> # 指定该作业的作业名;
--mail-type=<type> # 邮件通知, 触发参数 (NONE, BEGIN, END, FAIL, REQUEUE, ALL);
--mail-user=<user> # 发送给指定邮箱;
-n, --ntasks=<number> # 资源不满足时候,任务会暂停, --cpus-per-task参数可以修改该默认值;
-c, --cpus-per-task=<ncpus> # 每个任务所需要的核心数, 默认为1;
--ntasks-per-node=<ntasks> # 每个节点的任务数, --ntasks参数的优先级高于该参数, 如果使用--ntasks这个参数, 那么将会变为每个节点最多运行的任务数;
-o, --output=<filename pattern> # 输出文件, 作业脚本中的输出将会输出到该文件;
-p, --partition=<partition_names> # 将作业提交到对应分区;
-q, --qos=<qos> # 指定QOS;
-t, --time=<time> # 允许作业运行的最大时间,

sbatch条件任务

```
$ sbatch --ntasks=1 --time=10 work1.sbatch
```

Submitted batch job 1001

```
$ sbatch --ntasks=128 --time=60 --dependency=afterok:1001 work2.sbatch
```

Submitted batch job 1002

```
$ sbatch --ntasks=1 --time=30 --dependency=afterok:45002 work3.sbatch
```

Submitted batch job 1003

```
$ sbatch --begin=17:00:00 -n1 -t10 nightwork.bash
```

Submitted batch job 1004

可用选项

--ntasks or -n 任务线程，默认值是申请到的节点总CPU核数

--time or -t 最长运行时间此处单位是分钟 "hours:minutes:seconds","days-hours:minutes:seconds"

--dependency or -d 任务条件

--begin 延迟任务在某一时间执行

参考网站: <https://slurm.schedmd.com/sbatch.html>

练习

编写一个最近工作需要的SLURM任务脚本,使用sbatch 命令提交到64G24c分区